

MODUL 5 – Histogram, Histogram Equalization, dan Dithering

A. TUJUAN

1. Mahasiswa mampu memahami mengenai histogram citra dan menerapkannya dalam program Python
2. Mahasiswa mampu memahami mengenai *histogram equalization* dan menerapkannya dalam program Python
3. Mahasiswa mampu memahami mengenai dithering citra dan menerapkannya dalam program Python
4. Mahasiswa mampu menganalisis histogram citra hasil akuisisinya sendiri dan menyimpulkan kondisi pencahayaan saat citra tersebut diambil
5. Mahasiswa mampu membandingkan ekualisasi histogram global dengan CLAHE serta menentukan parameter yang paling sesuai untuk citranya, disertai alasan
6. Mahasiswa mampu menghitung secara manual satu langkah error diffusion Floyd-Steinberg dan memverifikasi hasilnya dengan program

B. ALAT DAN BAHAN

1. PC/LAPTOP
2. Github
3. *Google Colaborator*
4. Smartphone dengan kamera
5. Citra hasil akuisisi Pertemuan 2: KTM1a.jpg, KTM1b.jpg, KTM1c.jpg, KTM1d.jpg
6. Google Drive untuk penyimpanan citra pribadi

C. KETENTUAN PENGUMPULAN

Seluruh percobaan pada modul ini dikerjakan dua tahap: lebih dulu dengan citra contoh untuk memastikan kode Anda sudah benar, kemudian dengan citra KTM milik Anda sendiri sebagai bahan analisis. Parameter percobaan diturunkan dari NIM Anda, sehingga setiap mahasiswa bekerja pada nilai yang berbeda.

A. Citra Pribadi

Lakukan akuisisi dengan ketentuan berikut ini, setelah akuisisi lakukan cropping sehingga tidak ada background pada ktp anda:

Nama berkas	Isi	Dipakai pada
KTM1a.jpg	KTM/KTP difoto di dalam ruangan gelap	E-1, E-2, E-3
tilKTM1b.jpg	KTM/KTP dengan pencahayaan lampu ruangan	E-1, E-3
KTM1c.jpg	KTM/KTP di dalam ruangan dengan flash	E-1
KTM1d.jpg	KTM/KTP di luar ruangan dengan cahaya matahari	E-1, E-2

Selain citra KTM di atas, Anda juga memerlukan citra contoh lena.jpg dan lena_lc.jpg yang sudah disediakan pada percobaan kali ini. Citra contoh dipakai hanya untuk memastikan kode Anda sudah benar, sedangkan citra KTM dipakai sebagai bahan analisis dan penilaian.

Catatan privasi: KTM memuat data pribadi. Apabila repositori GitHub Anda bersifat publik, tutup area foto dan NIM pada citra yang ditampilkan di bagian yang dapat diakses publik, dan simpan citra asli pada Google Drive yang tidak dibagikan ke publik.

B. Parameter Pribadi Berbasis NIM

Nilai parameter pada seluruh percobaan tidak dipilih sembarangan, melainkan dihitung dari tiga digit terakhir NIM Anda yang disebut N. Dengan cara ini setiap mahasiswa bekerja pada titik operasi yang berbeda, dan setiap angka pada laporan dapat dihitung ulang oleh dosen.

Simbol	Rumus	Rentang	Dipakai pada
N	tiga digit terakhir NIM	0 - 999	dasar seluruh parameter
bins	$2^{\text{pangkat}((N \bmod 4) + 3)}$	8, 16, 32, 64	E-1, jumlah bin histogram
clip	$1,0 + (N \bmod 5) \times 0,5$	1,0 - 3,0	E-2, clipLimit CLAHE
tile	$(N \bmod 4) + 2$	2 - 5	E-2, tileGridSize CLAHE
L	$2^{\text{pangkat}((N \bmod 3) + 1)}$	2, 4, 8	E-3, jumlah level kuantisasi

C. Sel Identitas (Wajib Menjadi Sel Pertama)

Sel berikut wajib menjadi sel pertama notebook Week5, dijalankan, dan keluarannya ikut tersimpan pada berkas yang dikumpulkan. Notebook tanpa sel ini tidak dinilai.

```
# ===== SEL IDENTITAS - JANGAN DIHAPUS, HARUS SEL PERTAMA =====
import sys, platform, hashlib, datetime, zoneinfo

NAMA = 'Tulis nama lengkap Anda'
NIM = '2141720000' # ganti dengan NIM Anda

N = int(NIM[-3:])
PARAM = {
    'bins' : 2 ** ((N % 4) + 3),
    'clip' : round(1.0 + (N % 5) * 0.5, 1),
    'tile' : (N % 4) + 2,
    'L' : 2 ** ((N % 3) + 1),
}
waktu = datetime.datetime.now(zoneinfo.ZoneInfo('Asia/Jakarta'))

print('NAMA :', NAMA)
print('NIM :', NIM, '| N =', N)
for k, v in PARAM.items():
    print('%-6s : ' % k, v)
print('WAKTU :', waktu.strftime('%Y-%m-%d %H:%M:%S %Z'))
print('SISTEM :', sys.version.split()[0], platform.platform())
kunci = NIM + '|' + waktu.strftime('%Y-%m-%d')
print('TOKEN :', hashlib.sha256(kunci.encode()).hexdigest()[:16])
```

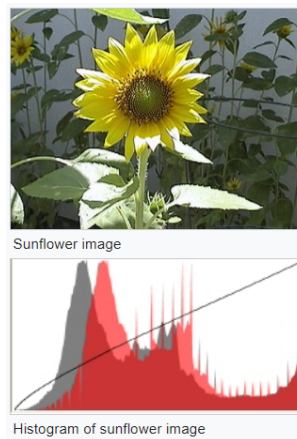
Setiap citra keluaran yang Anda tampilkan wajib diberi judul yang memuat NIM, contoh:

```
plt.title(NIM + ' - CLAHE clip ' + str(PARAM['clip']))
```

D. ULASAN TEORI

D.1 Histogram Image

Citra Histogram adalah tipe histogram yang berfungsi sebagai representasi grafis dari distribusi intensitas warna pixel pada citra digital. Grafik histogram seperti pada gambar 1 akan menampilkan jumlah pixel dari tiap intensitas warna pixel. Dengan memperhatikan grafik histogram, seseorang dapat menilai distribusi warna pixel secara cepat. Untuk gambar berwarna histogram mewakili setiap channel warna yang ada, sedangkan untuk citra grayscale hanya terdiri dari satu histogram saja. Axis horizontal dari grafik histogram mewakili variasi nilai pixel, sedangkan axis vertikalnya mewakili jumlah pixel. Sebelah kiri dari axis horizontal mewakili area hitam dan gelap, bagian tengah mewakili area abu-abu, dan bagian kanan mewakili area terang dan putih.



Gambar 1. Contoh Citra Asli dan Histogram Citra Tersebut

Berikut adalah algoritma dalam membuat histogram dari citra digital

1	Siapkan dua form, form 1 berisi 1 chart yang digunakan untuk menampilkan histogram citra grayscale, form 2 berisi 3 chart yang digunakan untuk menampilkan histogram citra R, G, dan B.
2	Buat variable penyimpan histogram R, G, dan B Variabel dapat disimpan dalam array 2 dimensi atau menggunakan Dictionary pada Visual Studio. Tipe data untuk axis horizontal dapat menggunakan int atau Byte, sedangkan pada axis vertical, disarankan menggunakan tipe data Double. Hal ini dilakukan karena nilai histogramnya akan kita normalisasi. Pada percobaan ini data histogram kita normalisasi karena selanjutnya akan dilakukan proses perbaikan citra menggunakan histogram. Percobaan perbaikan citra menggunakan histogram akan kita lakukan pada jobsheet berikutnya.
3	Buat variable bertipe Bitmap untuk menyimpan nilai citra masukan
4	Beri nilai awal untuk tiap intensitas pixel dengan nilai 0
5	Untuk setiap baris dan kolom citra, periksa nilai pixel dan tambahkan 1 untuk tiap nilai pixel yang diperiksa
6	Periksa nilai dictionary, jika histogram dari R sama persis dengan G //Tampilkan form 1
7	Simpan nilai histogram dari dictionary ke dalam variable <i>kunci</i> bertipe data List
8	Tampilkan form 1
9	Untuk setiap nilai kunci
10	Normalisasi nilai histogram
11	Tambahkan data histogram ke dalam chart
12	Selesai
13	Jika tidak sama antara R dan G //Tampilkan form 2
14	Lakukan langkah yang sama seperti ketika menampilkan form 1
15	selesai

Catatan: tabel algoritma di atas ditulis untuk lingkungan Visual Studio dengan objek Form, Chart, Bitmap, dan Dictionary. Pada praktikum ini seluruh percobaan dikerjakan dengan Python, sehingga algoritmanya menjadi sebagai berikut.

```
import numpy as np, cv2 as cv, matplotlib.pyplot as plt

def histogram_manual(img, L=256):
    h = np.zeros(L, dtype=np.int64)
    for y in range(img.shape[0]):
        for x in range(img.shape[1]):
            h[img[y, x]] += 1
    return h

# 1. siapkan penghitung, isi 0
# 2. telusuri setiap baris
# dan setiap kolom
# 3. tambah 1 sesuai nilai piksel

gray = cv.imread(BASE + '/KTM1b.jpg', cv.IMREAD_GRAYSCALE)
h = histogram_manual(gray)
p = h / gray.size
cdf = np.cumsum(p)
assert h.sum() == gray.size

# 4. normalisasi, jumlahnya = 1
# 5. distribusi kumulatif
# pemeriksaan wajib

# citra berwarna: ulangi untuk tiap kanal
bgr = cv.imread(BASE + '/KTM1d.jpg')
for i, nama in enumerate(['Biru', 'Hijau', 'Merah']):
    plt.plot(histogram_manual(bgr[:, :, i]), label=nama)
```

Perulangan bersarang sengaja ditulis agar alurnya terlihat. Untuk citra berukuran besar, gunakan np.histogram atau cv.calcHist yang jauh lebih cepat, dan buktikan hasilnya sama persis.

D.2 Histogram Equalization

Histogram Equalization adalah persamaan yang diimplementasikan pada sebuah citra *digital* di mana distribusi histogramnya akan lebih menyebar, dalam hal ini walaupun tidak dapat

dibuktikan bahwa bentuk histogram-nya akan seragam namun dengan *Histogram Equalization* dapat dipastikan histogram-nya akan lebih merata. Perataan histogram diperoleh dengan mengubah derajat keabuan sebuah piksel (r) dengan derajat keabuan yang baru (s) dengan sebuah fungsi transformasi (T) (Gonzalez & Woods, 2002). *Histogram Equalization* dilakukan dengan membuat transform function dari histogram image yang telah dinormalisasi. Berikut adalah persamaan untuk transform function:

$$G(z_q) = (L - 1) \sum_{i=0}^q p_z(z_i)$$

Perhitungan manual untuk *Histogram Equalization* kali ini menggunakan 3-bit image ($L = 8$) dengan size 64x64 piksel ($MN = 4096$) dan distribusi intensitas akan diperlihatkan pada Tabel 1, dimana level intensitas adalah berupa nilai integer pada jarak $[0, L - 1] = [0, 7]$.

Gambar 2 menunjukkan proses perbaikan citra digital menggunakan *Histogram Equalization*.



Gambar 2. Flowchart jalannya perbaikan citra digital dengan HE

Pada Gambar 3 ditunjukkan original histogram sebelum dilakukan proses ekualisasi. Pada Gambar 4 menunjukkan hasil transform function Pada Gambar 5 ditunjukkan histogram hasil penghitungan dengan metode HE. Penghitungan *Histogram Equalization* menggunakan rumus:

$$s_0 = T(r_0) = 7 \sum_{j=0}^1 p_r(r_j) = 7 p_r(r_0) = 1.33$$

$$s_1 = T(r_1) = 7 \sum_{j=0}^1 p_r(r_j) = 7 p_r(r_0) + 7 p_r(r_1) = 3.08$$

dan $s_2 = 4.55$, $s_3 = 5.67$, $s_4 = 6.23$, $s_5 = 6.65$, $s_6 = 6.86$, $s_7 = 7.0$

r_k	n_k	$Pr(r_k) = n_k/MN$
$r_{0=0}$	790	0.19
$r_{1=1}$	1023	0.25
$r_{2=2}$	850	0.21
$r_{3=3}$	656	0.16
$r_{4=4}$	329	0.08
$r_{5=5}$	245	0.06
$r_{6=6}$	122	0.03
$r_{7=7}$	81	0.02

Tabel 1 Distribusi intensitas dan nilai histogram untuk 3-bit image, 64 x 64 citra digital

Dimana n_k = nilai piksel k , dan n adalah seluruh jumlah piksel pada suatu citra. Rumus lain yang dapat digunakan untuk menghitung *histogram equalization* adalah sebagai berikut :

$$K_o = \text{round} \left(\frac{C_i \cdot (2^k - 1)}{w \cdot h} \right)$$

Dimana :

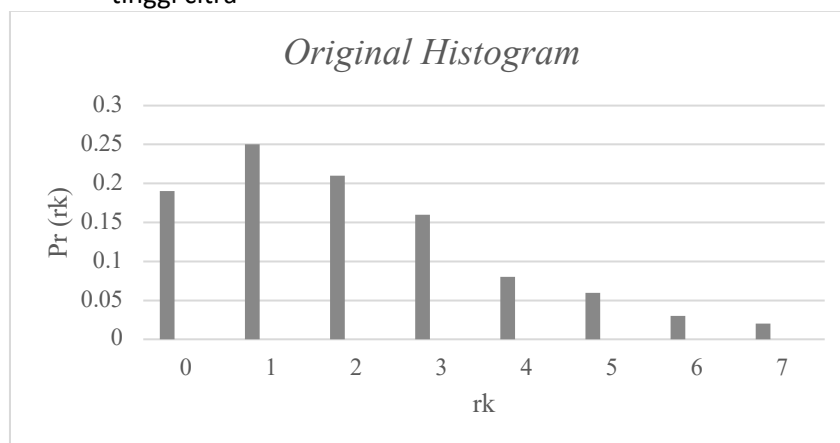
C_i = Distribusi Kumulatif nilai pixel

round = Fungsi pembulatan terdekat

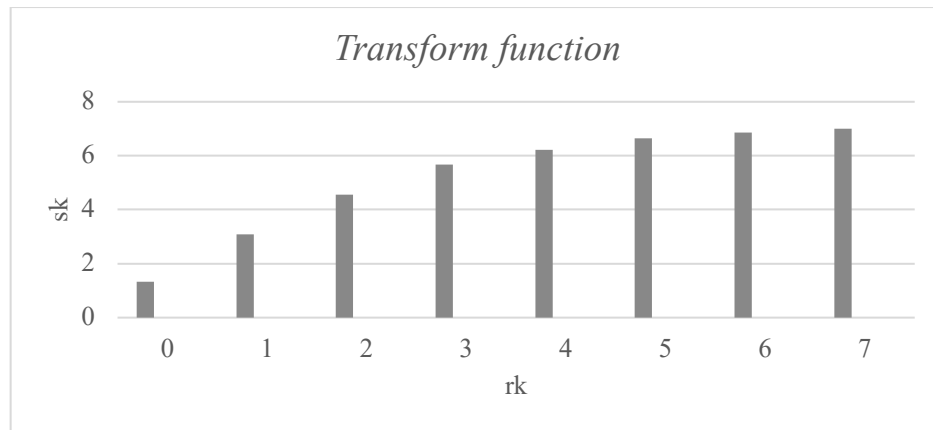
K_o = Nilai intensitas warna pixel hasil *Histogram Equalization*

w = lebar citra

h = tinggi citra



Gambar 3. Original histogram sebelum dilakukan proses ekualisasi



Gambar 4. Transform function



Gambar 5. Histogram hasil penghitungan dengan metode *Equalization Histogram*

Pada point ini, s memiliki nilai berbentuk pecahan maka nilai tersebut dibulatkan pada nilai integer yang paling dekat.

$$s_0 = 1.33 \rightarrow 1$$

$$s_0 = 6.23 \rightarrow 6$$

$$s_1 = 3.08 \rightarrow 3$$

$$s_0 = 6.65 \rightarrow 7$$

$$s_2 = 4.55 \rightarrow 5$$

$$s_0 = 6.86 \rightarrow 7$$

$$s_3 = 5.67 \rightarrow 6$$

$$s_0 = 7.00 \rightarrow 7$$

Rangkuman perhitungan di atas disajikan ulang pada tabel berikut. Perhatikan penulisan pada daftar di atas: kolom sebelah kanan seharusnya bernama s4 sampai s7, bukan s0 seluruhnya.

rk	p(rk)	CDF(rk)	sk = 7 x CDF	sk dibulatkan
r0 = 0	0,19	0,19	1,33	1
r1 = 1	0,25	0,44	3,08	3
r2 = 2	0,21	0,65	4,55	5
r3 = 3	0,16	0,81	5,67	6
r4 = 4	0,08	0,89	6,23	6
r5 = 5	0,06	0,95	6,65	7
r6 = 6	0,03	0,98	6,86	7
r7 = 7	0,02	1,00	7,00	7

Delapan level lama menyusut menjadi enam level baru, yaitu 1, 3, 5, 6, dan 7. Nilai 0, 2, dan 4 tidak pernah muncul pada citra hasil, sedangkan r4 dan r3 sama-sama menjadi 6. Inilah sebabnya histogram hasil ekualisasi pada citra digital tidak pernah benar-benar rata.

D.3 Image Dithering (Error Diffusion)

Error Diffusion adalah salah satu tipe halftoning dimana proses kuantisasi didistribusi pada piksel tetangga yang belum diproses. Penggunaan utamanya adalah untuk mengubah kedalaman image menjadi lebih kecil, tetapi kualitas gambar tidak terlalu buruk. Proses ini biasanya digunakan pada teknologi cetak, karena jumlah warna tinta yang tidak terlalu banyak sehingga ilusi warna terbentuk dari teknik ini.

Error diffusion digolongkan sebagai area operation, karena yang dilakukan oleh error diffusion pada satu pixel akan mempengaruhi hasil perhitungan operasi pixel lainnya. Error diffusion juga memiliki kemampuan untuk memperkuat tepi dari citra. Hal ini dapat membuat teks dalam citra menjadi lebih mudah terbaca daripada teknik halftoning yang lain.

Beberapa metode Error Diffusion diantaranya adalah:

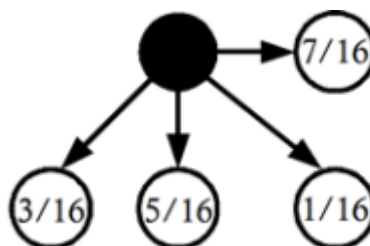
1. Floyd and Steinberg

Floyd dan Steinberg mendeskripsikan system pembentuk error diffusion pada citra digital dengan menggunakan kernel sederhana:

$$\frac{1}{16} \begin{bmatrix} - & \# & 7 \\ 3 & 5 & 1 \end{bmatrix}$$

Dimana – menunjukkan pixel yang sudah diproses, # menunjukkan pixel saat ini yang sedang diproses.

Atau dalam bentuk diagram node ditunjukkan dengan Gambar 6 berikut:



Gambar 6. Diagram Node Dithering Floyd and Steinberg

Rumus dithering Floyd and Steinberg memiliki persamaan sebagai berikut :

$$\text{Rumus pertama (R1)} \rightarrow I_{acc}(i+1, j) = I_{acc}(i+1, j) + \frac{7}{16}e$$

$$\text{Rumus pertama (R2)} \rightarrow I_{acc}(i+1, j+1) = I_{acc}(i+1, j+1) + \frac{3}{16}e$$

$$\text{Rumus pertama (R3)} \rightarrow I_{acc}(i, j+1) = I_{acc}(i, j+1) + \frac{5}{16}e$$

$$\text{Rumus pertama (R4)} \rightarrow I_{acc}(i-1, j+1) = I_{acc}(i-1, j+1) + \frac{1}{16}e$$

Dimana :

I_{acc} = Target pixel (pixel yang sedang diproses)

i = baris

j = kolom

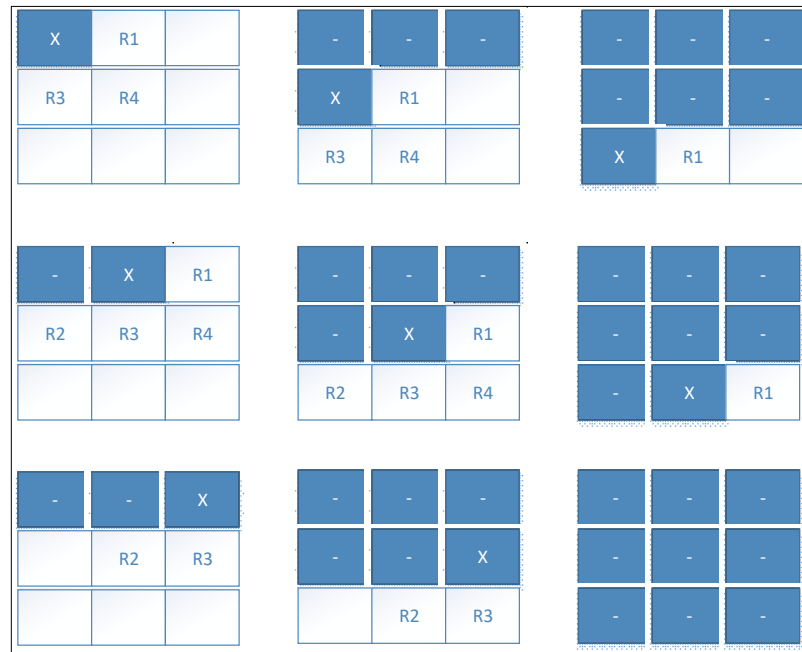
e = error

Catatan konvensi sumbu. Pada daftar di atas i dinyatakan sebagai baris dan j sebagai kolom, sehingga penulisan $I(i+1, j)$ berarti berpindah satu baris ke bawah. Padahal bobot 7/16 pada Floyd-Steinberg diberikan kepada tetangga di sebelah kanan, bukan di bawah. Agar tidak membingungkan, seluruh praktikum modul ini memakai satu konvensi: x adalah kolom dan y adalah baris, dengan pemindaian dari kiri ke kanan lalu baris atas ke bawah.

Tetangga	Posisi terhadap piksel yang diproses	Bobot
$I(x+1, y)$	kanan, satu kolom setelahnya	7/16
$I(x-1, y+1)$	baris bawah, satu kolom sebelumnya	3/16
$I(x, y+1)$	tepat di bawahnya	5/16
$I(x+1, y+1)$	baris bawah, satu kolom setelahnya	1/16
Jumlah	$7 + 3 + 5 + 1 = 16$	1

Pseudo-code Floyd-Steinberg pada akhir sub bab ini sudah memakai konvensi tersebut, sehingga pseudo-code itulah yang dipakai sebagai acuan bila terdapat perbedaan penulisan.

Misalkan saja, untuk citra berukuran 3 x 3 pixel, maka dithering dilakukan sesuai dengan tahapan rumus yang sesuai dengan posisi yang akan dihitung seperti gambar 7 berikut.



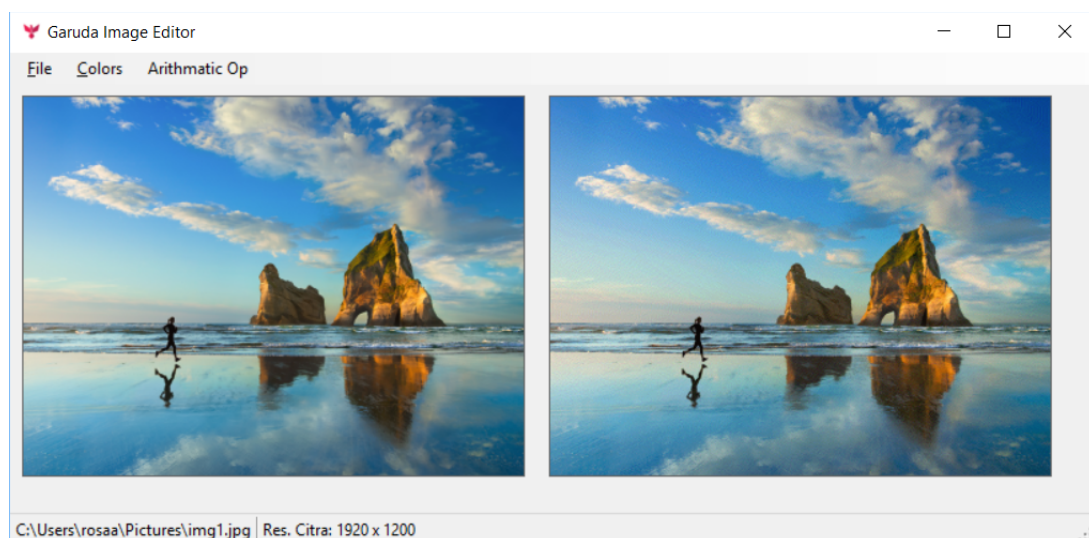
Gambar 7. Posisi Implementasi rumus Dithering Floyd and Steinberg

- Jarvis, Judice dan Ninke dari Bell Labs mendeskripsikan system yang sama yang mereka istilahkan “minimized average error”, menggunakan kernel dengan ukuran yang lebih besar:

$$\frac{1}{48} \begin{bmatrix} - & - & \# & 7 & 5 \\ 3 & 5 & 7 & 5 & 3 \\ 1 & 3 & 5 & 3 & 1 \end{bmatrix}$$

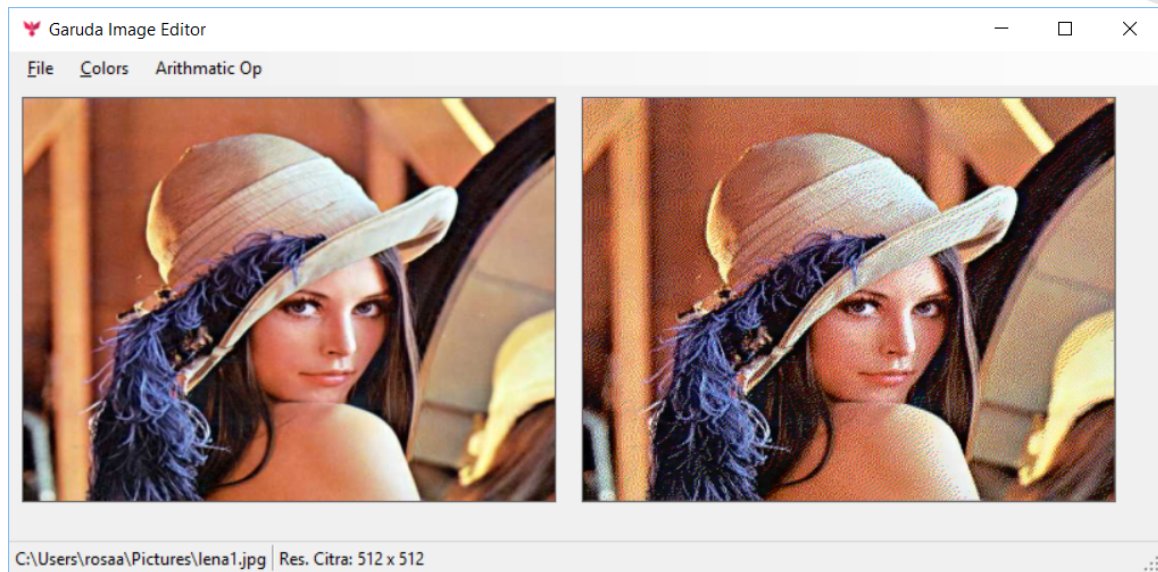
Jika ingin menyederhanakan jumlah warna tanpa error diffusion, maka cara yang termudah adalah dengan mencari warna terdekatnya. Jika input warna lebih dekat ke merah, maka warnanya akan diubah kemerah. Jika lebih dekat ke cyan, maka warnanya diubah ke cyan, dan seterusnya. Hasilnya terlihat pada gambar pada sub bab warna terdekat.

Perhatikan gambar 8 berikut sebelum dan sesudah disederhanakan kedalam 8 warna.



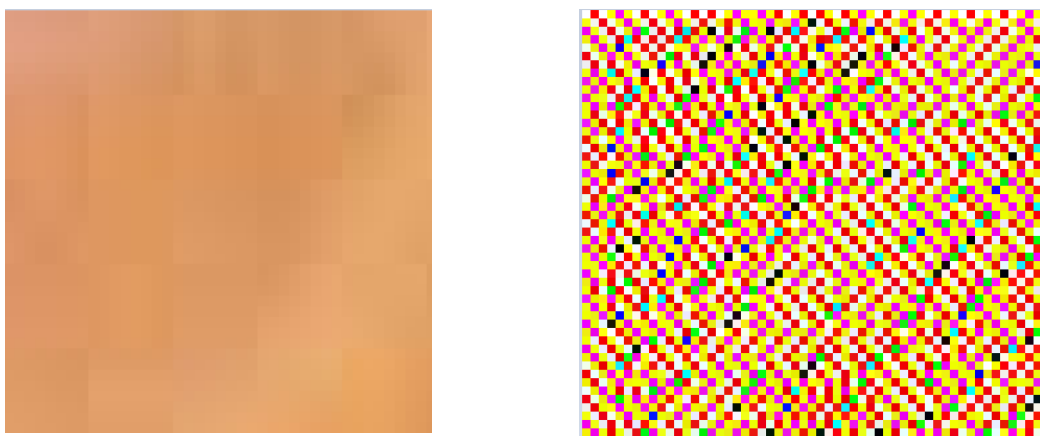
Gambar 8. Penyederhanaan Warna Gambar Menjadi 8 Warna

Berikut adalah contoh yang lain:



Gambar 9. Contoh lain Penyederhanaan Warna Gambar Menjadi 8 Warna

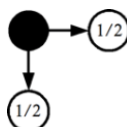
Dari gambar tersebut diambil bagian pojok kiri atas dan kemudian diperbesar sebagai berikut :



Gambar 10. Hasil Zoom Penyederhanaan Gambar Menjadi 8 Warna

Error diffusion bekerja dengan membandingkan warna asli pixel dengan warna terdekat yang ditentukan dan menghitung selisihnya. Selisih ini dinamakan dengan error. Porsi error ini kemudian dibagi ke pixel tetangga sehingga menyebabkan errornya terdifusi dan dinamakan “Error Diffusion”.

Bentuk paling sederhana dari error diffusion terlihat dari gambar berikut:



Gambar 11. Bentuk Sederhana Error Diffusion

Dengan bentuk error diffusion seperti diatas, setengah error dari pixel yang diproses (ditunjukkan dengan titik hitam) terdifusi setengahnya ke pixel sebelah kanan dan setengahnya lagi ke pixel bawahnya. Pada error diffusion warna, proses ini harus dilakukan disemua channel red, green, dan blue.

Bagian penting yang perlu digarisbawahi adalah jumlah total error yang terdifusi tidak boleh melebihi satu. Hal penting lain yang perlu dicatat adalah ketika porsi error terdifusi ke tetangganya, tetangga tersebut memiliki nilai pixel antara 0 – 255. Jika nilainya diluar 0 – 255, maka nilai akhir perlu di truncate.

Berikut adalah pseudo-code untuk error difusi sederhana yang harus dilakukan setelah sebuah pixel(x,y) diubah nilainya ke warna terdekat:

1	Error = warna asli - warnaTerdekat
2	SetPixel(x+1, y) = Truncate(GetPixel(x+1,y) + 0.5 * error)
3	SetPixel(x, y+1) = Truncate(GetPixel(x, y+1) + 0.5 * error)

Berikut adalah pseudo-code untuk error difusi Floyd and Steinberg yang harus dilakukan setelah sebuah pixel(x,y) diubah nilainya ke warna terdekat:

1	Error = warna asli - warnaTerdekat
2	SetPixel(x+1, y) = Truncate(GetPixel(x+1, y) + 7/16 * error)
3	SetPixel(x-1, y+1) = Truncate(GetPixel(x-1, y+1) + 3/16 * error)
4	SetPixel(x , y+1) = Truncate(GetPixel(x , y+1) + 5/16 * error)
5	SetPixel(x+1, y+1) = Truncate(GetPixel(x+1, y+1) + 1/16 * error)

Catatan implementasi. Simpan citra kerja sebagai float sebelum proses difusi error, misalnya `img.astype(np.float32)`. Pada tipe `uint8` penjumlahan error akan berputar (240 + 30 menjadi 14) sehingga hasilnya berbintik. Nilai tetangga tetap dijaga pada rentang 0 sampai 255 dengan `np.clip`, dan tetangga yang berada di luar batas citra dilewati. Untuk citra berwarna, proses ini dijalankan terpisah pada kanal B, G, dan R.

```
def floyd_steinberg(gray, L=2):
    img = gray.astype(np.float32).copy()
    step = 255.0 / (L - 1)
    H, W = img.shape
    for y in range(H):
        for x in range(W):
            lama = img[y, x]
            baru = np.round(lama / step) * step      # warna palet terdekat
            img[y, x] = baru
            error = lama - baru
            if x + 1 < W:      img[y, x+1] += error * 7 / 16
            if x > 0 and y+1 < H:  img[y+1, x-1] += error * 3 / 16
            if y + 1 < H:      img[y+1, x ] += error * 5 / 16
            if x + 1 < W and y+1 < H:  img[y+1, x+1] += error * 1 / 16
    return np.clip(img, 0, 255).astype(np.uint8)
```

E. PERCOBAAN PRAKTIKUM

Cara mengerjakan percobaan pada modul ini

Setiap percobaan dikerjakan dalam dua tahap.

Tahap 1, verifikasi dengan citra contoh. Jalankan kode Anda pada citra contoh yang disebutkan, yaitu `lena.jpg` atau `lena_lc.jpg`, lalu bandingkan outputnya dengan gambar contoh

pada modul. Keluaran Anda harus sama. Bila berbeda, perbaiki kode lebih dulu sebelum melanjutkan.

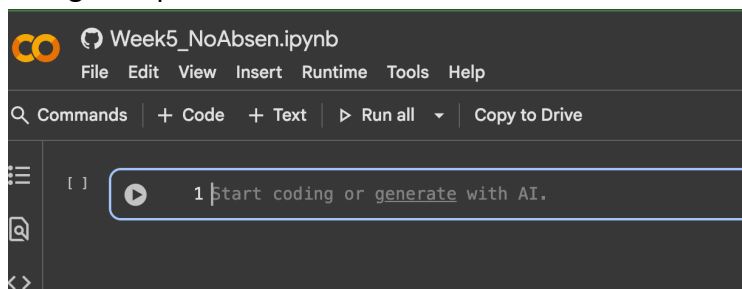
Tahap 2, penerapan pada citra sendiri. Ulangi percobaan yang sama pada citra KTM hasil akuisisi Anda pada Pertemuan 2. Hasil tahap inilah yang dianalisis dan dinilai.

Tampilkan keluaran kedua tahap pada notebook dan beri judul yang memuat NIM pada setiap citra. Tetapkan dua variabel folder berikut sekali saja, lalu pakai di seluruh percobaan.

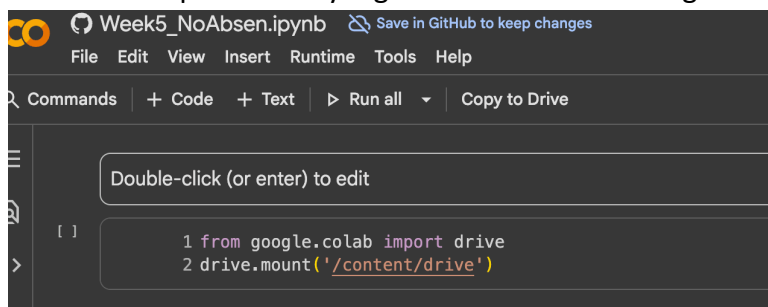
CONTOH = '/content/drive/MyDrive/PCVK/Images'	# citra contoh bersama
BASE = '/content/drive/MyDrive/PCVK/Images/' + NIM	# citra KTM milik Anda

E-1 PERCOBAAN HISTOGRAM

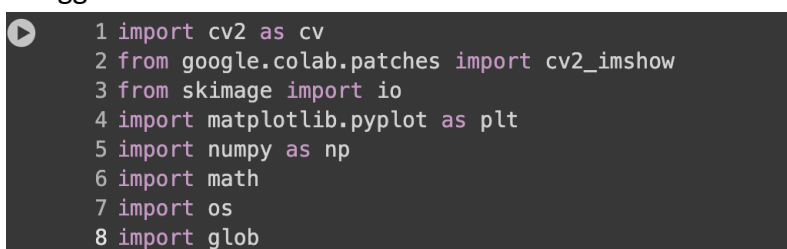
1. Buka <https://colab.research.google.com/>. Setelah dipastikan bahwa Google Colab terhubung dengan GitHub Anda, lanjutkan dengan memilih repository yang telah digunakan pada praktikum minggu lalu, lalu ubah nama berkas menjadi "Week5_<No Absen>.ipynb". Salin sel identitas dari subbab Ketentuan Pengumpulan sebagai sel pertama notebook.



Kemudian import folder yang ada di Drive Anda dengan cara sebagai berikut.

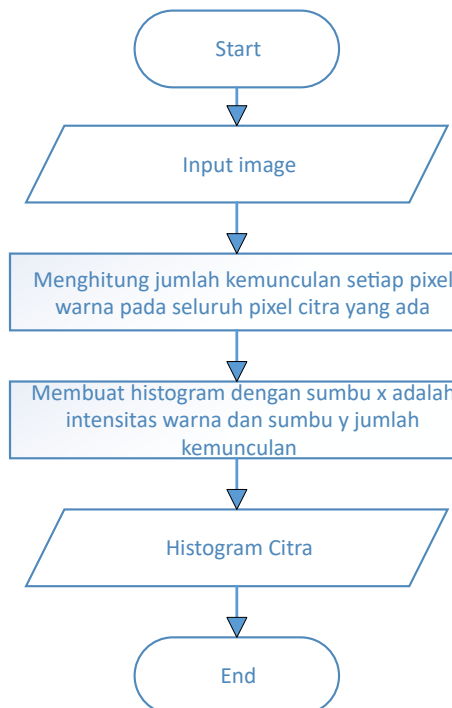


2. Import beberapa library berikut yang akan digunakan selama uji coba praktikum minggu ke-5 berikut.



3. Membuat histogram citra berdasarkan flowchart di bawah ini. Tahap 1 gunakan citra contoh lena.jpg dan cocokkan keluarannya dengan gambar contoh pada modul; setelah cocok, tahap 2 ulangi dengan citra KTM1b.jpg milik Anda sendiri.

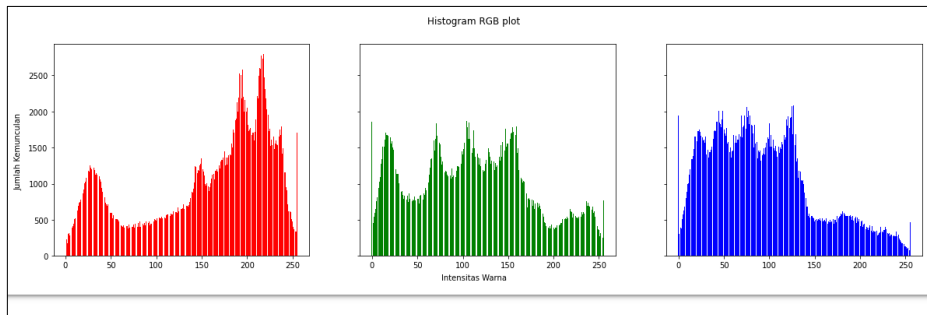
Gambar berikut adalah keluaran yang harus Anda peroleh pada tahap 1 dengan citra contoh. Bila keluaran Anda berbeda, perbaiki kode lebih dulu sebelum melanjutkan ke tahap 2 dengan citra KTM Anda.



```

1 #membuat histogram image (manual)
2 img = cv.imread('/content/drive/MyDrive/Colab Notebooks/PCVK_2024/Image Jobsheet/lena.jpg')
3 img = cv.cvtColor(img, cv.COLOR_BGR2RGB)
4
5 height, width, depth = np.shape(img)
6 names = np.arange(256)
7
8 red = [0]*256
9 green = [0]*256
10 blue = [0]*256
11
12 for y in range(0,height) :
13     for x in range(0,width) :
14         red[img[y][x][0]] += 1
15         green[img[y][x][1]] += 1
16         blue[img[y][x][2]] += 1
17
18
19 names = np.arange(256)
20 fig, axs = plt.subplots(1, 3, figsize=[20,5], sharex=True, sharey=True)
21 fig.suptitle('Histogram RGB plot')
22 fig.text(0.09, 0.5, 'Jumlah Kemunculan', va='center', rotation='vertical')
23 fig.text(0.5, 0.04, 'Intensitas Warna', ha='center')
24 axs[0].bar(names, red, color='red')
25 axs[1].bar(names, green, color='green')
26 axs[2].bar(names, blue, color='blue')
    
```

Output :



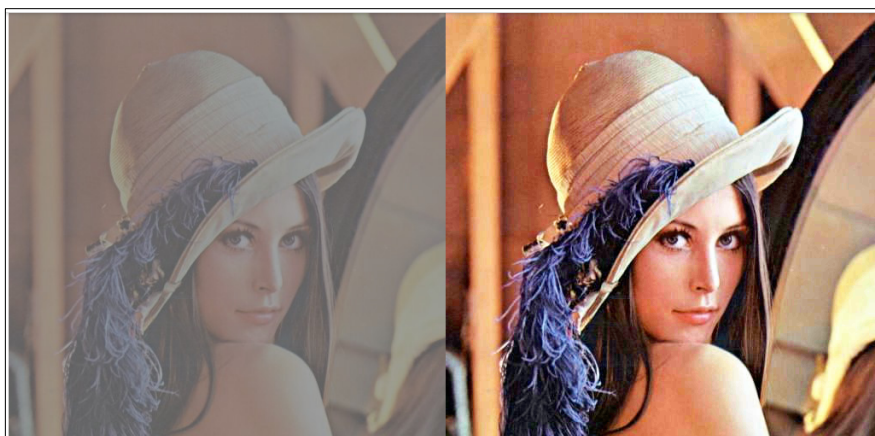
PERTANYAAN PRAKTIKUM E1

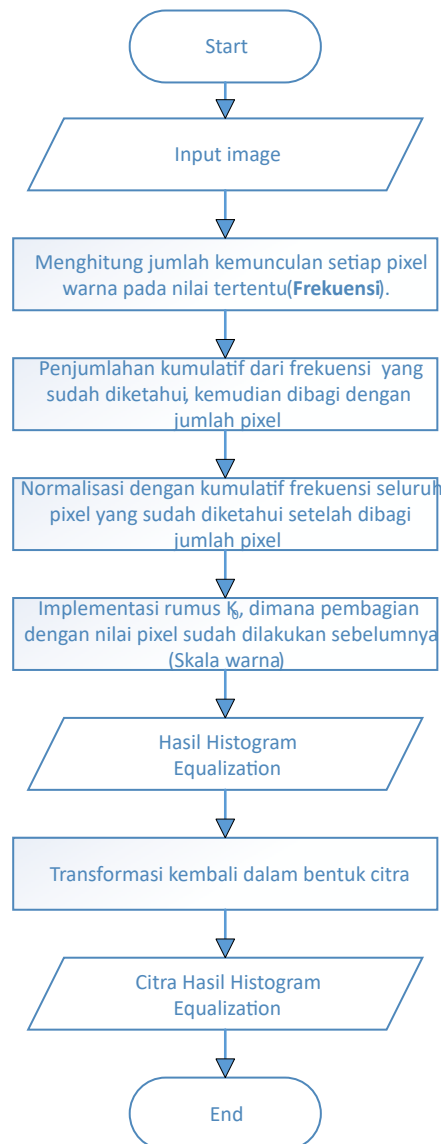
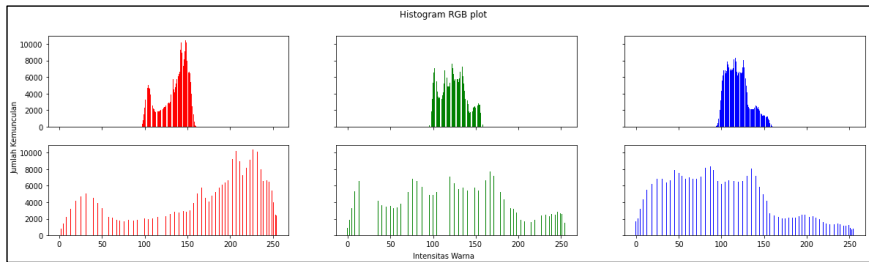
1. Buatlah histogram citra yang sama menggunakan library NumPy, yaitu `numpy.histogram`, dan juga `cv.calcHist`. Bandingkan ketiganya dengan menghitung selisih maksimum antar hasil. Selisihnya harus nol; tampilkan pembuktiannya pada `lena.jpg` lebih dulu, kemudian ulangi pada `KTM1b.jpg`.
2. Gambarkan histogram ternormalisasi $p(j)$ dan kurva CDF untuk `KTM1a.jpg` sampai `KTM1d.jpg` dalam satu figure 2 x 2. Jelaskan perbedaan bentuk kurva CDF antara citra paling gelap dan citra paling terang milik Anda, lalu kaitkan dengan kondisi pencahayaan saat akuisisi pada Pertemuan 2.
3. Ulangi histogram `KTM1b.jpg` dengan jumlah bin sebesar `PARAM['bins']` dan dengan 256 bin. Tampilkan keduanya berdampingan dan jelaskan informasi apa yang hilang ketika jumlah bin dikurangi.

E-2 PERCOBAAN HISTOGRAM EQUALIZATION

1. Buatlah histogram equalization beserta tampilan citra sebelum dan sesudah proses, berdasarkan flowchart di bawah ini. Tahap 1 gunakan citra contoh `lena_lc.jpg` dan cocokkan hasilnya dengan gambar contoh pada modul; tahap 2 ulangi pada citra `KTM1a.jpg` milik Anda, yaitu hasil akuisisi di ruangan gelap.

Gambar berikut adalah keluaran yang harus Anda peroleh pada tahap 1 dengan citra contoh. Bila keluaran Anda berbeda, perbaiki kode lebih dulu sebelum melanjutkan ke tahap 2 dengan citra KTM Anda.





- Setelah mengerjakan langkah no. 1, buatlah histogram citra yang sama akan tetapi menggunakan library yang dimiliki oleh CV2 yaitu “equalizeHist” seperti pada potongan kode berikut ini.

```

b, g, r = cv2.split(image)

b_equalized = cv2.equalizeHist(b)
g_equalized = cv2.equalizeHist(g)
r_equalized = cv2.equalizeHist(r)
    
```

Lengkapi potongan kode tersebut. Bandingkan hasil `equalizeHist` dengan hasil implementasi manual Anda dengan menghitung selisih maksimum antara keduanya, baik pada `lena_lc.jpg` maupun pada `KTM1a.jpg`, lalu jelaskan penyebab selisih tersebut apabila tidak nol.

3. Ekualisasi juga dapat diterapkan pada citra berwarna. Cara yang biasanya pertama kali terpikirkan adalah mengekualisasi kanal biru, hijau, dan merah satu per satu. Jalankan potongan kode berikut pada citra contoh `lena.jpg` terlebih dulu, kemudian ulangi pada `KTM1d.jpg` milik Anda, dan amati warna yang dihasilkan pada keduanya.

```
import cv2 as cv, numpy as np, matplotlib.pyplot as plt

berkas = CONTOH + '/lena.jpg'          # tahap 1; tahap 2 ganti: BASE + '/KTM1d.jpg'
bgr     = cv.imread(berkas)

# cara 1: tiap kanal diekualisasi sendiri-sendiri
kanal = list(cv.split(bgr))
for i in range(3):
    kanal[i] = cv.equalizeHist(kanal[i])
he_rgb = cv.merge(kanal)
```

4. Cara kedua memisahkan kecerahan dari informasi warna. Citra diubah ke ruang warna YCrCb, hanya kanal Y yang diekualisasi, lalu dikembalikan ke BGR. Kanal Cr dan Cb yang menyimpan informasi warna tidak disentuh sama sekali. Kanal V pada HSV dan kanal L pada LAB berperan serupa, sehingga ketiganya dicoba sekaligus. (gunakan `KTM1d.jpg`)

```
# cara 2: hanya kanal terang yang diekualisasi
ycrcb = cv.cvtColor(bgr, cv.COLOR_BGR2YCrCb)
y, cr, cb = cv.split(ycrcb)
he_y = cv.cvtColor(cv.merge([cv.equalizeHist(y), cr, cb]), cv.COLOR_YCrCb2BGR)

# varian setara pada ruang warna lain
hsv = cv.cvtColor(bgr, cv.COLOR_BGR2HSV)
h, sat, v = cv.split(hsv)
he_v = cv.cvtColor(cv.merge([h, sat, cv.equalizeHist(v)]), cv.COLOR_HSV2BGR)

lab = cv.cvtColor(bgr, cv.COLOR_BGR2LAB)
l, a, b = cv.split(lab)
he_l = cv.cvtColor(cv.merge([cv.equalizeHist(l), a, b]), cv.COLOR_LAB2BGR)
```

Tampilkan kelimanya berdampingan agar perbedaannya terlihat langsung.

```
judul = ['Asli', 'Ekualisasi B, G, R', 'Kanal Y', 'Kanal V', 'Kanal L']
citra = [bgr, he_rgb, he_y, he_v, he_l]

plt.figure(figsize=(18, 4))
for i, (c, t) in enumerate(zip(citra, judul), 1):
    plt.subplot(1, 5, i)
    plt.imshow(cv.cvtColor(c, cv.COLOR_BGR2RGB))
    plt.title(NIM + ' - ' + t, fontsize=9)
    plt.axis('off')
plt.show()
```

Perbedaan kedua cara ini tidak cukup dinilai dengan mata. Ukur pergeseran warnanya dengan membandingkan kanal Hue dengan citra asli. Kanal Hue menyimpan jenis warna, sehingga nilai yang

berubah banyak berarti warna citra memang bergeser. Perhatikan bahwa Hue pada OpenCV bersifat melingkar pada rentang 0 sampai 179, jadi selisihnya dihitung lewat jalur terpendek.

```
def geser_hue(asli, hasil):
    h1 = cv.cvtColor(asli, cv.COLOR_BGR2HSV)[: , : , 0].astype(np.int16)
    h2 = cv.cvtColor(hasil, cv.COLOR_BGR2HSV)[: , : , 0].astype(np.int16)
    d = np.abs(h1 - h2)
    d = np.minimum(d, 180 - d)          # Hue melingkar 0-179
    return d.mean()

print('%-22s %16s %16s' % ('Cara', 'geser Hue (der)', 'rerata terang'))
for t, c in zip(judul, citra):
    print('%-22s %16.2f %16.1f' % (t, geser_hue(bgr, c) * 2,
                                         cv.cvtColor(c, cv.COLOR_BGR2GRAY).mean()))
```

Catat hasilnya pada tabel berikut.

Cara ekualisasi	Rata-rata pergeseran Hue (derajat)	Rerata kecerahan sesudah	Catatan pengamatan visual
Citra asli	-		
Ekualisasi kanal B, G, R			
Kanal Y (YCrCb)			
Kanal V (HSV)			
Kanal L (LAB)			

Jawab pertanyaan berikut berdasarkan angka pada tabel Anda, bukan hanya dari pengamatan mata:

1. Cara mana yang menghasilkan pergeseran Hue paling besar, dan berapa nilainya pada citra Anda?
2. Mengapa ekualisasi kanal terang tetap menghasilkan pergeseran Hue yang tidak persis nol? Kaitkan jawaban Anda dengan pembulatan dan pemotongan nilai ketika citra dikembalikan ke ruang warna BGR.
3. Di antara kanal Y, V, dan L, mana yang paling sesuai untuk foto KTM Anda? Sebutkan alasannya dan tunjukkan bagian citra yang menjadi dasar penilaian.
4. Ulangi salah satu cara pada kanal terang dengan mengganti cv.equalizeHist menjadi CLAHE yang memakai clipLimit = PARAM['clip'] dan tileGridSize = (PARAM['tile'], PARAM['tile']). Bandingkan pergeseran Hue dan rerata kecerahannya dengan hasil ekualisasi biasa.

PERTANYAAN PRAKTIKUM E2

1. Ekualisasi global pada citra KTM Anda

- a. Terapkan ekualisasi manual dan cv.equalizeHist pada KTM1a.jpg versi grayscale.
- b. Tampilkan citra asli, citra hasil ekualisasi, serta histogram dalam satu layout.
- c. Hitung PSNR antara citra asli dan citra hasil ekualisasi. Jelaskan mengapa nilai PSNR justru turun padahal citra terlihat lebih jelas, lalu simpulkan apakah PSNR layak dipakai untuk menilai keterbacaan sebuah citra.

2. Ekualisasi lokal dengan CLAHE pada citra KTM Anda

- a. Terapkan CLAHE pada KTM1a.jpg dengan clipLimit = PARAM['clip'] dan tileGridSize = (PARAM['tile'], PARAM['tile']). Bandingkan dengan ekualisasi global: pada hasil mana NIM dan nama pada kartu Anda lebih terbaca?
- b. Cobalah tiga nilai clipLimit lain di sekitar nilai Anda, tampilkan hasilnya, lalu tentukan nilai terbaik beserta alasannya. Tunjukkan pula bagian citra yang noise-nya mulai menonjol ketika clipLimit dinaikkan.

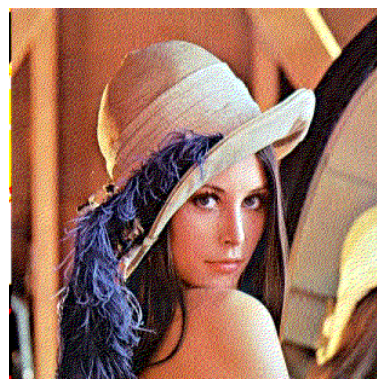
E-3 TUGAS PRAKTIKUM *DITHERING*

1. Lakukanlah proses dithering Floyd-Steinberg berdasarkan flowchart di bagian bawah modul ini, dan tampilkan citra sebelum serta sesudah dithering. Tahap 1 gunakan lena.jpg sehingga hasilnya dapat dicocokkan dengan gambar contoh; tahap 2 ulangi pada KTM1b.jpg dengan jumlah level PARAM['L'].

Gambar berikut adalah keluaran yang harus Anda peroleh pada tahap 1 dengan citra contoh. Bila keluaran Anda berbeda, perbaiki kode lebih dulu sebelum melanjutkan ke tahap 2 dengan citra KTM Anda.



Citra asli lena



citra setelah dithering

2. Ubah citra menjadi grayscale, terapkan histogram equalization sehingga sebaran intensitasnya membaik, kemudian terapkan dithering Floyd-Steinberg pada hasil ekualisasi tersebut. Kerjakan lebih dulu pada lena_lc.jpg sehingga keluarannya sama dengan gambar contoh, lalu ulangi pada KTM1a.jpg. Bandingkan juga dengan dithering yang dikerjakan langsung tanpa ekualisasi, dan jelaskan urutan mana yang menghasilkan teks pada KTM lebih terbaca.

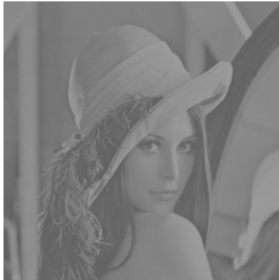
2141720042 - lena.jpg asli



2141720042 - Floyd-Steinberg, L = 2



2141720042 - lena_jc.jpg asli



2141720042 - dithering langsung

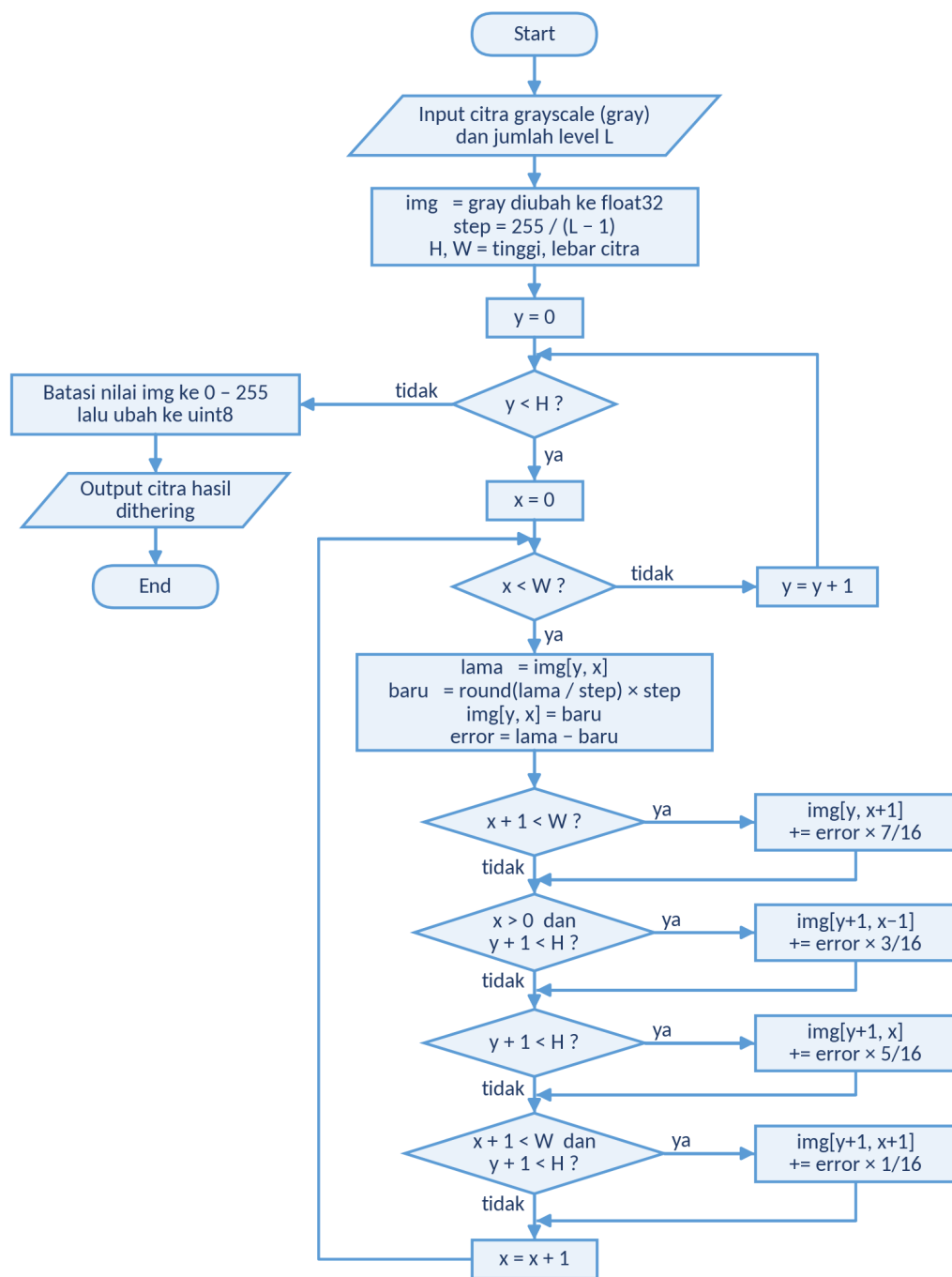


2141720042 - hasil ekualisasi



2141720042 - ekualisasi lalu dithering





Flowchart dithering Floyd and Steinberg

Catatan : Untuk citra berwarna, fungsi `floyd_steinberg` pada flowchart yang sama cukup dijalankan tiga kali, sekali untuk tiap kanal B, G, dan R. Setelah itu ketiga kanal digabung lagi.

--- SELAMAT BELAJAR ---